

1 Motivation and Business Case

Theme: cloud-based production-engineering education.

1.1 Problem Statement and Formulation

Students and junior engineers need opportunities to practise responding to production failures before they become responsible for live services. Reading an incident report explains what happened, but does not require the reader to investigate incomplete evidence or decide how to recover a service. In a real incident, a decision can also change the problem itself. An attempted fix may provide temporary relief, create another failure, or make recovery harder.

The learning problem therefore includes both diagnosis and judgement. Learners must decide which signals matter, identify a likely cause, and predict how an intervention will affect the system. They must then assess the result and revise their approach. Google's Wheel of Misfortune supports this practice through team exercises that reenact incidents from previous postmortems [1]. Google's guidance also recommends reviewing drill outcomes to identify lessons and gaps [2].

1.2 Purpose, Target Users, and Proposed Value

OpsReplay is a production-engineering learning platform with three modes: **Learn**, **Challenges**, and **Code Review**. Its primary users are computer science students, along with junior software, DevOps, platform, and site reliability engineers. Learn explains documented failures. Code Review develops the ability to identify risky changes. Challenges place learners in a live, deliberately misconfigured environment to investigate and recover.

OpsReplay adapts this training approach for individual practice in a web browser. Each Challenge runs real services in an isolated cloud container. Learners diagnose with production tools, such as `curl`, `dig`, and service logs, and repair real configuration files. Consequences follow from real system behaviour, and automated checks confirm recovery without a team exercise.

For example, a web application may return errors because a DNS record still points to a retired server. Fixing the record does not help until the reverse proxy reloads, because the proxy resolved the name at startup. A faulty zone edit followed by a proxy restart takes the whole site offline. After an attempt, learners can play back their session beside the logs and metrics to see how each action affected the system, or retry in a fresh environment. The intended benefit is stronger diagnosis and recovery judgement, which Section 4 will assess.

2 Business Model

OpsReplay uses a freemium model across all three modes:

- **Free:** a limited set of Learn entries, Challenges, and Code Review exercises, mostly at Easy tier. Free Challenges include the full debrief and session playback, so users experience the core workflow before paying.
- **Pro:** a monthly subscription that unlocks the expanded set in every mode, including Medium and Hard exercises, more Code Review languages, and longer Challenge sessions.
- **Institutional:** Pro access licensed per seat to a class or training cohort.

The paid value is the authored practice experience. Limits matter most for Challenges, because each session runs its own container. Content development, container compute, and language-model use are the main costs. Prices and willingness to pay remain unvalidated. The pilot will assess learning value and interest in the expanded set, while system measurements will estimate the cost of delivering a session.

3 Architecture and Implementation

All three modes share one web application, account system, and progress store. Challenges and Code Review have Easy, Medium, and Hard tiers, reflecting evidence ambiguity, service dependencies, and defect subtlety.

3.1 Learn

Learn is a curated library of documented failures. Each entry gives a category, a team-written summary, lessons, a link to the official incident report, and related exercises. Entries are written in Markdown with metadata for category and access plan. A build step renders free entries as static pages served through CloudFront, while Pro entries stay in private S3 and the API returns them only after an entitlement check.

3.2 Challenges

Environment. Challenges are single-player and the main development focus. Each is a versioned container image with a small service stack, such as nginx, a web application, PostgreSQL, and CoreDNS, plus one planted misconfiguration. Starting a Challenge launches a dedicated Fargate task holding two containers. The challenge container runs the services under a process supervisor and hosts the learner’s terminal. A monitor container generates steady user traffic, collects metrics and logs, and runs validation checks. It reaches the services over the task’s shared localhost network, and keeping it separate stops learners altering measurements.

Investigation and repair. Each attempt begins with an alert. Learners use a root shell in a browser terminal with standard tools, such as `curl`, `dig`, `ss`, and `psql`, alongside service logs and configuration files. A dashboard shows request rate, errors, latency, and resource use, with commands marked on the timeline. There is no fixed action list: any valid fix restores service, and mistakes have real effects. The gateway records the terminal stream, and the monitor container records service logs, metrics, and changed configuration files after each command, so every step can be reviewed.

Recovery and failure. Validators, such as end-to-end checkouts succeeding for 60 consecutive seconds, define recovery. The attempt succeeds when all pass and fails at the time limit. Traffic continues throughout, so failed requests accumulate while the fault remains. Table 1 shows the proposed initial Challenges.

Challenge	Planted misconfiguration	Why action order matters
Wrong upstream port (Easy)	nginx <code>proxy_pass</code> targets port 8081, but the app now listens on 8080, so users see 502 errors.	Editing <code>nginx.conf</code> without <code>nginx -t</code> risks a syntax error. A reload fails safely, but a restart stops the proxy.
Stale DNS record (Medium)	The CoreDNS zone maps <code>api.internal</code> to the retired API address, which nginx cached at startup.	The fix needs an nginx reload. Restarting nginx while the zone fails to load takes the site offline.
Connection exhaustion (Hard)	A missing index slows queries. Gunicorn <code>workers</code> $\times$ SQLAlchemy <code>pool_size</code> nears PostgreSQL <code>max_connections</code> .	More workers exhaust connections and break a job worker on the same database. The index fixes the cause.

Table 1: Proposed Challenges. Each uses real services and configuration files.

Scoring and debrief. Scoring considers time to recovery, cumulative failed requests measured by the monitor container, investigation effort, and self-inflicted outages. Reduced impact counts even when the root cause remains. The debrief aligns the recorded commands with the metrics to explain key and missed evidence, action effects, the root cause, and recommended recovery.

Session playback. After the debrief, learners can play back the attempt as a timeline. The terminal recording runs in step with the logs, metrics, and configuration changes captured at each command, with key evidence and harmful actions highlighted. Learners can also retry the Challenge in a fresh task. Retries are labelled separately and never change the first attempt or its score.

3.3 Code Review

Each exercise is a bundle in private S3 with a diff, context, language, tier, and reference findings with line ranges and explanations. Learners flag risky lines in the rendered diff and write a short concern for each. On submission, a Lambda handler matches flagged lines against the reference ranges, reports found and missed issues, and reveals the explanations. Written concerns are shown beside the reference and not graded. Learners filter by language and tier. Some diffs introduce faults that later appear in Challenges, such as the pool setting in Table 1.

3.4 AI Teaching Support

An optional large language model (LLM) assistant supports Challenges and Code Review debriefs. In a Challenge, each call receives recent commands and output, visible metrics, and released hints. The model explains output and concepts and may propose a command, which runs only after the learner confirms it. The planted fault, validators, and unreleased hints stay outside the prompt. The LLM never sets scores or grades free text. AI and hint use are recorded to distinguish assisted results, and every mode still works if the provider fails.

3.5 Proposed AWS Deployment

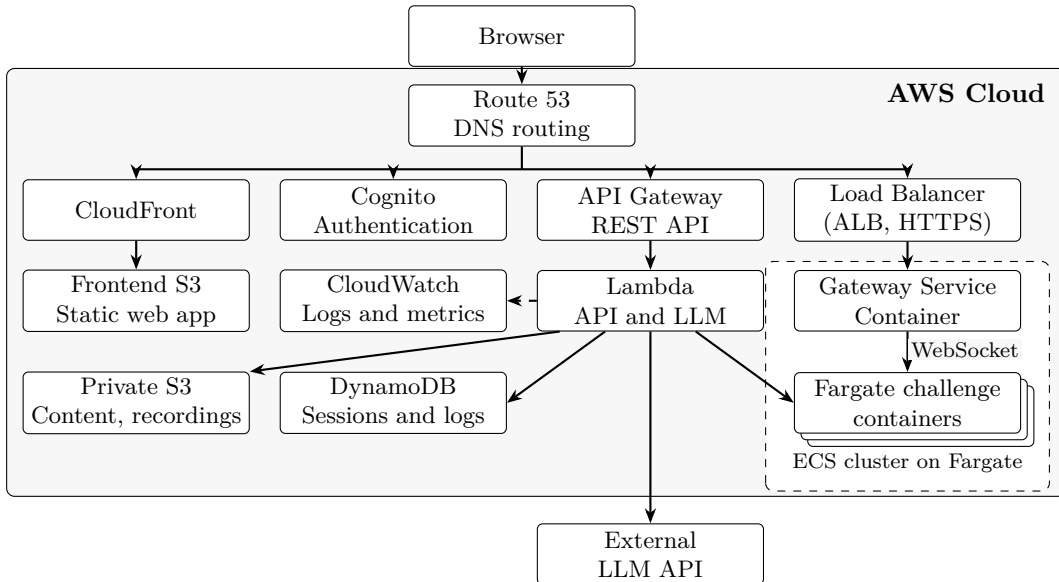


Figure 1: Proposed deployment. Solid arrows show requests and the dashed arrow shows monitoring.

Entry and web tier. Figure 1 shows the proposed single-Region deployment. Route 53 hosts the OpsReplay domain and maps each subdomain to its entry point with alias records: the web app to CloudFront, the API to API Gateway, terminals to an Application Load Balancer (ALB), and sign-in to Cognito. Route 53 health checks can fail over to a static maintenance endpoint. CloudFront serves the web app and free Learn pages from S3. Cognito identifies users, and the backend checks session ownership and plan entitlement on every request.

API and state. A Regional API Gateway REST API routes requests to stateless Lambda handlers. The LLM route streams responses, which REST APIs support, while the cheaper HTTP API lacks streaming and has a 30-second integration limit [3]. Lambda stays billed while waiting on the model, even after a client disconnects [4], so provider timeouts, token limits, and a separate concurrency budget bound AI use. DynamoDB stores progress, sessions, per-command timeline events, review submissions, and conversation history, all read by key. Terminal recordings and log captures exceed DynamoDB’s item size, so they go to private S3. Session start and end use conditional writes keyed by request ID, so a retry cannot launch a second task or record a result twice.

Challenge environments. To start a Challenge, a Lambda handler calls ECS RunTask with the Challenge’s task definition, pinned to an ECR image digest, and stores the task’s private address in DynamoDB. An EventBridge Scheduler job stops the task at the time limit, and a missed heartbeat stops abandoned tasks sooner. Terminal and dashboard traffic uses WebSockets. The gateway service

container is our own small WebSocket proxy, run as an ECS service in the same Fargate cluster as the challenge containers. It validates a short-lived session token, looks up the learner’s task address, proxies the terminal, and records the stream outside the learner’s reach. The ALB only terminates HTTPS and spreads connections across gateway copies; it does not balance challenge containers. A small warm pool can hide start-up time before a scheduled class.

Isolation. Each Fargate task has its own isolation boundary and shares no kernel, CPU, memory, or network interface with other tasks [5]. Environment tasks run in private subnets with no internet route, accept traffic only from the gateway service, and pull images through VPC endpoints. They have no task IAM role, so root shells expose no AWS credentials. CPU, memory, and time caps limit misuse.

Trade-offs. Per-session tasks add compute cost, start-up latency of tens of seconds, vCPU quotas that cap concurrent sessions, and AWS-specific integration. Fargate also disallows privileged containers, so environments cannot run nested Docker [5]. A single Docker host has fewer components but needs provisioned capacity and isolates learners’ root shells less strongly. The same images run under local Docker for testing and on any container host, which supports the on-premise comparison.

3.6 Implementation Plan

The team will prove one complete Challenge first, then a small Learn collection and Code Review set.

Period	Planned deliverable
21–28 Sep	Finalise the reference Challenge and submit the preliminary report.
29 Sep–11 Oct	Build the reference image, agent, validators, and session recording, and test them on AWS.
12–25 Oct	Integrate interface, dashboard, persistence, playback, and LLM. Add Learn and Code Review.
26 Oct–5 Nov	Validate additional Challenges and run system tests and the learner study.
6–13 Nov	Resolve findings and complete cost analysis, the final report, slides, and demonstration.

Table 2: Proposed 2026 delivery schedule, with Challenges receiving most development effort.

4 Evaluation Plan

Results will identify scenario image versions, deployment settings, and sample size.

4.1 System Correctness, Reliability, and Performance

- **Scenario validity.** Scripted runs of each image will confirm that the fault is present at start, validators fail before the reference fix and pass after it, and each trap in Table 1 behaves as stated. Twenty repeats per Challenge will expose flaky environments and measure recovery-time variance. Playback of each reference run must match its recorded output, logs, and metrics.
- **Isolation and access.** Test that tasks cannot reach the internet, other tasks, or AWS credentials, and that users cannot open another learner’s session or unentitled content. Retries, interrupted writes, and provider failures must not launch duplicate tasks or repeat score penalties. Every ended session must leave no running task.
- **Performance.** At 10, 50, and 100 concurrent sessions, measure time to a ready terminal with and without a warm pool, terminal round-trip latency, median and 95th-percentile API latency, errors, and DynamoDB throttling. Record action mix and think time. These are proposed loads, subject to Fargate quota.

4.2 LLM Accuracy, Latency, and Cost

A labelled request set will specify context and acceptable commands or explanations. Tests will measure suggestion accuracy, destructive suggestions, unsupported claims, answer leakage, and teaching accuracy. For each model, we will record median and 95th-percentile latency for the full conversational loop and token cost per session. Environment load tests will use simulated responses, labelled as such, with separate real-provider tests under a fixed budget.

4.3 User Evaluation

We propose a formative pilot with 5–10 computer science students or junior engineers. Each will investigate one Challenge, read its debrief, and review the session playback. For another Challenge of similar difficulty, the learner will read a static explanation of the evidence, actions, and outcomes. We will balance assignment and order where the sample permits, allow equal study time, and provide equivalent causal information. The AI assistant will be off, isolating the environment and debrief workflow.

Questions before and after each condition will assess diagnosis and causal reasoning with a common rubric. After each condition, learners will diagnose a short new scenario and justify a recovery plan, testing whether the lesson transfers. We will record completion, self-inflicted outages, and explanation quality. Brief tasks will check Learn, Code Review, and optional AI support. Participants will complete the System Usability Scale (SUS) [6] and a short interview on usability and playback.

Results will report individual scores, paired differences, and usability problems. The pilot gives early evidence to guide improvement, but cannot establish broad effectiveness, long-term retention, or the separate effect of playback within the full workflow.

4.4 Cloud Cost and On-Premise Comparison

On-demand containers and serverless handlers assume intermittent demand. We will compare steady use and class-sized bursts with an on-premise deployment of the same images, using measured workloads and published rates. AWS costs will include per-session Fargate vCPU and memory, API requests, Lambda, database, storage, transfer, authentication, and monitoring, plus the ALB, VPC endpoints, and gateway service, which incur cost even when idle. On-premise costs will include a container host sized for peak class load over its useful life, power, networking, maintenance, and staff time. Both include the same external LLM cost. We will report cost per completed session, capacity, and availability assumptions, with credits separate and no assumption that cloud hosting is cheaper.

5 Conclusion

OpsReplay combines free learning with authored practice in real, isolated environments, focused on live incidents with real consequences and session playback of recovery decisions. The team will prove one complete Challenge before expanding the library and evaluating usability, learning, reliability, and cost. The plan assumes access to testers, sufficient Fargate quota, and an LLM provider with function calling. Exercise counts, languages, scoring weights, model choice, and prices remain open.

References

- [1] Google, “Postmortem Culture: Learning from Failure,” *The Site Reliability Workbook*.
<https://sre.google/workbook/postmortem-culture/>.
- [2] Google, “Incident Response,” *The Site Reliability Workbook*.
<https://sre.google/workbook/incident-response/>.
- [3] AWS, “Choose Between REST APIs and HTTP APIs” and “Quotas for HTTP APIs,” *API Gateway Developer Guide*.
- [4] AWS, “Stream the Integration Response” and “Response Streaming for Lambda Functions,” *AWS Documentation*.
- [5] AWS, “Amazon ECS on AWS Fargate” and “Fargate Security Considerations for Amazon ECS,” *Amazon ECS Developer Guide*.
- [6] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, 1996, pp. 189–194.

Web references accessed 21 September 2026; [5] accessed 24 September 2026.

AI-use declaration. The team developed the project concept and selected the design direction. AI assisted with design evaluation, reference checks, report drafting, language refinement, and $\LaTeX$ preparation. This work clarified the distinction between repeatable simulation results and learning outcomes that require user evaluation.